

FIT2099 Assignment 3 Design Document [REQ 1-5]

MA_AppliedSession07_B

Yang Li Shen, 36031542

Edwin Chuah Wei-Hann, 35981229

Lingsha Deng, 35894342

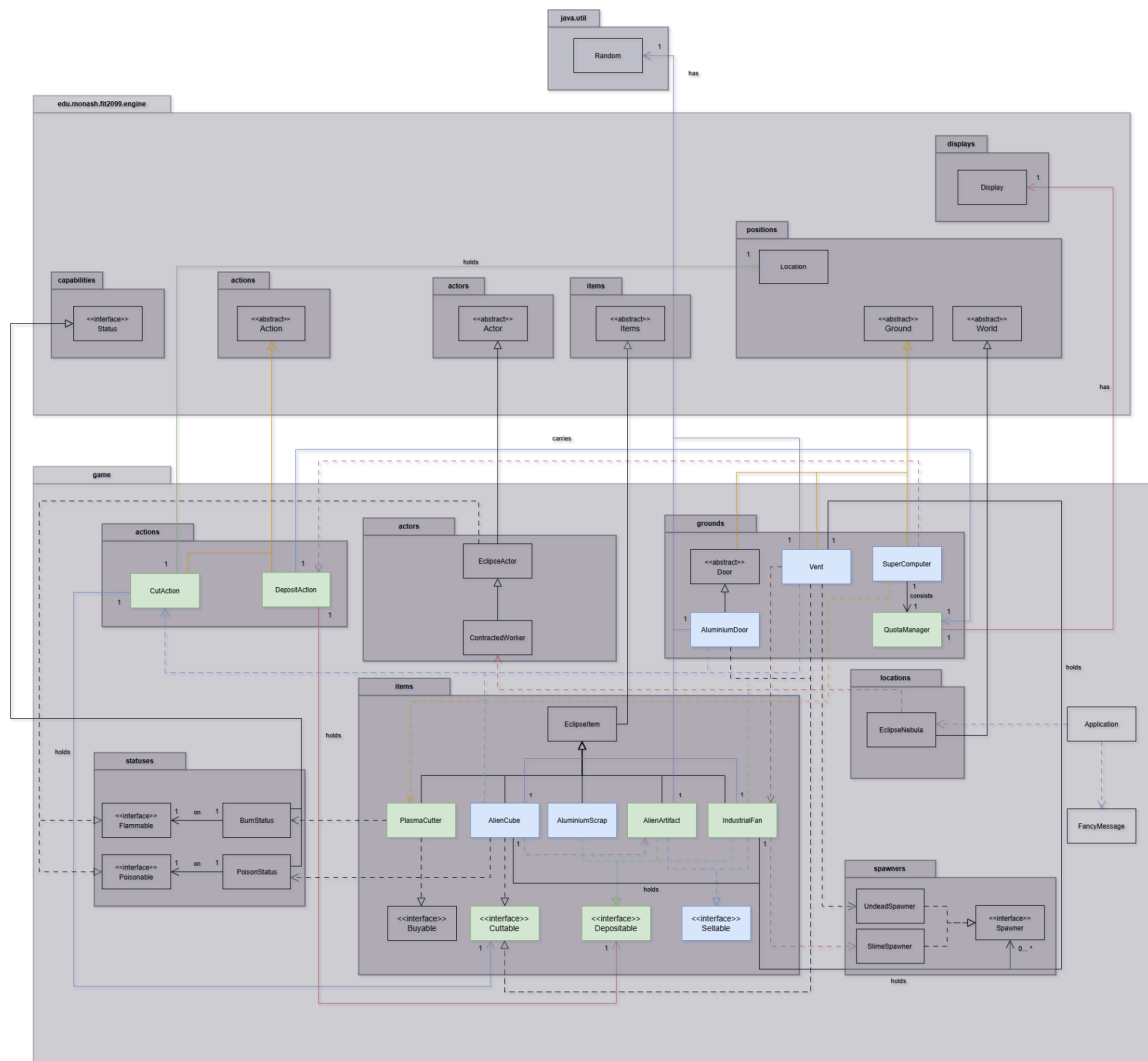
Edmond Cheong, 36367729

Erwyna Soo Wen Xin, 36555789

Design Diagrams

D. REQ 1 (Edmond Cheong, 36367729)

UML



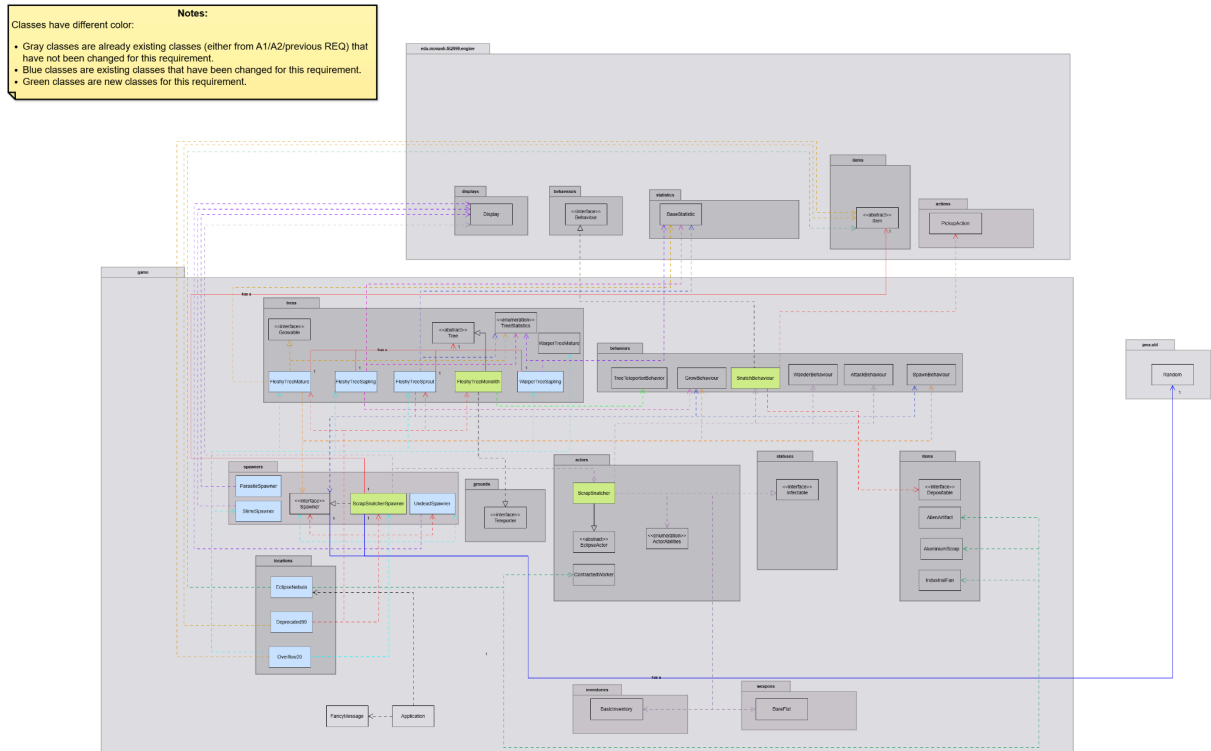
Note:

Legend

Green represents: classes that are new
 Blue represents: classes that existed but has been modified
 White/Default represents: classes that has no changes to it

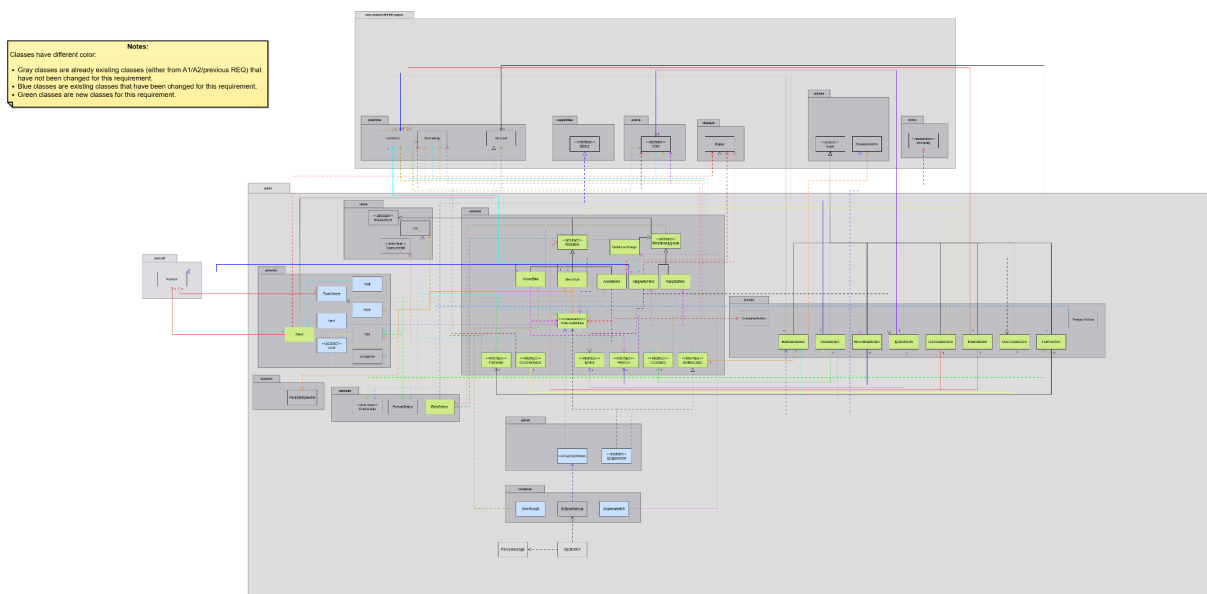
D. REQ 2 (Lingsha Deng, 35894342)

UML



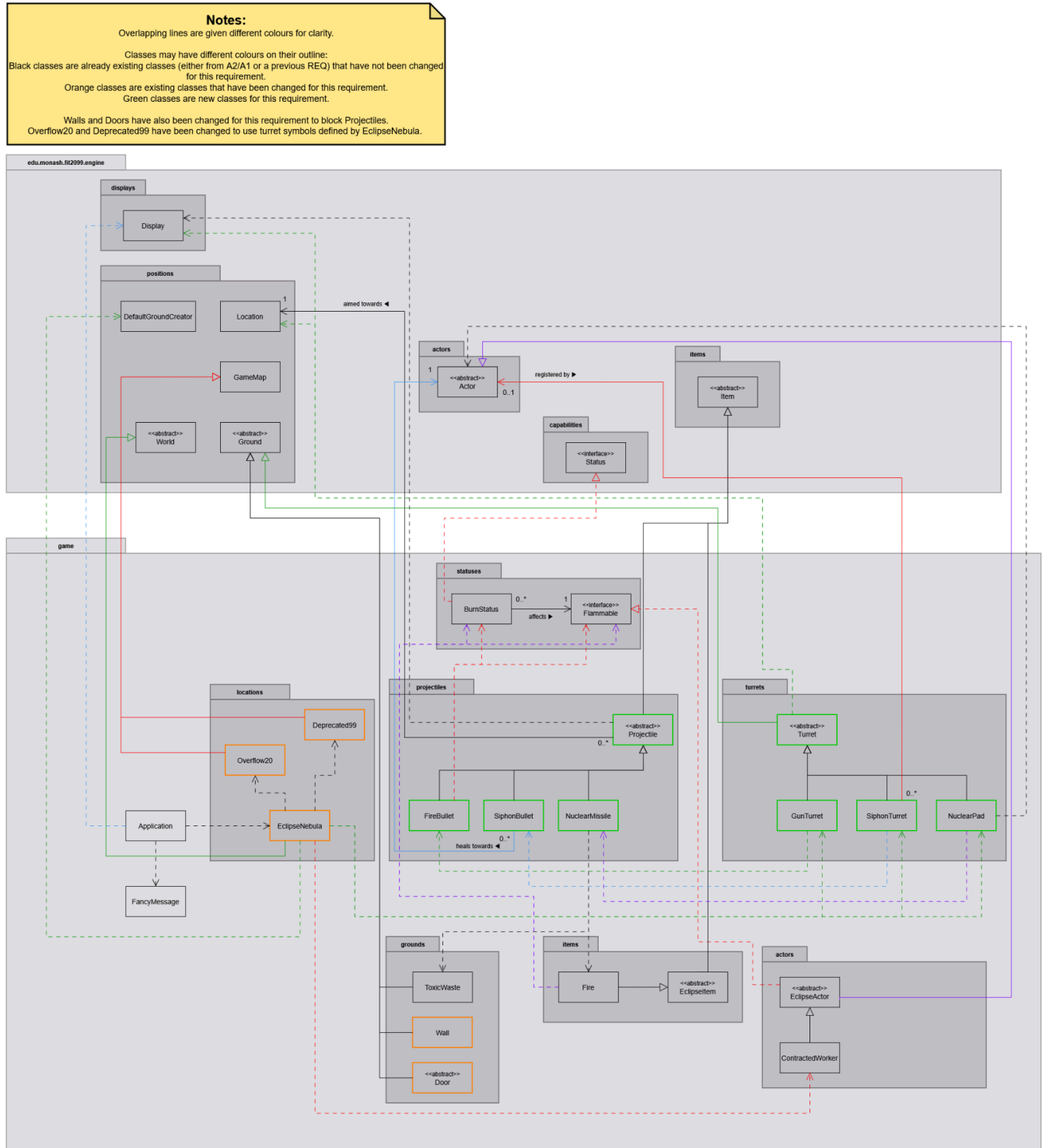
D. REQ 3 (Yang Li Shen, 36031542)

UML



D. REQ 4 (Edwin Chuah Wei-Hann, 35981229)

UML

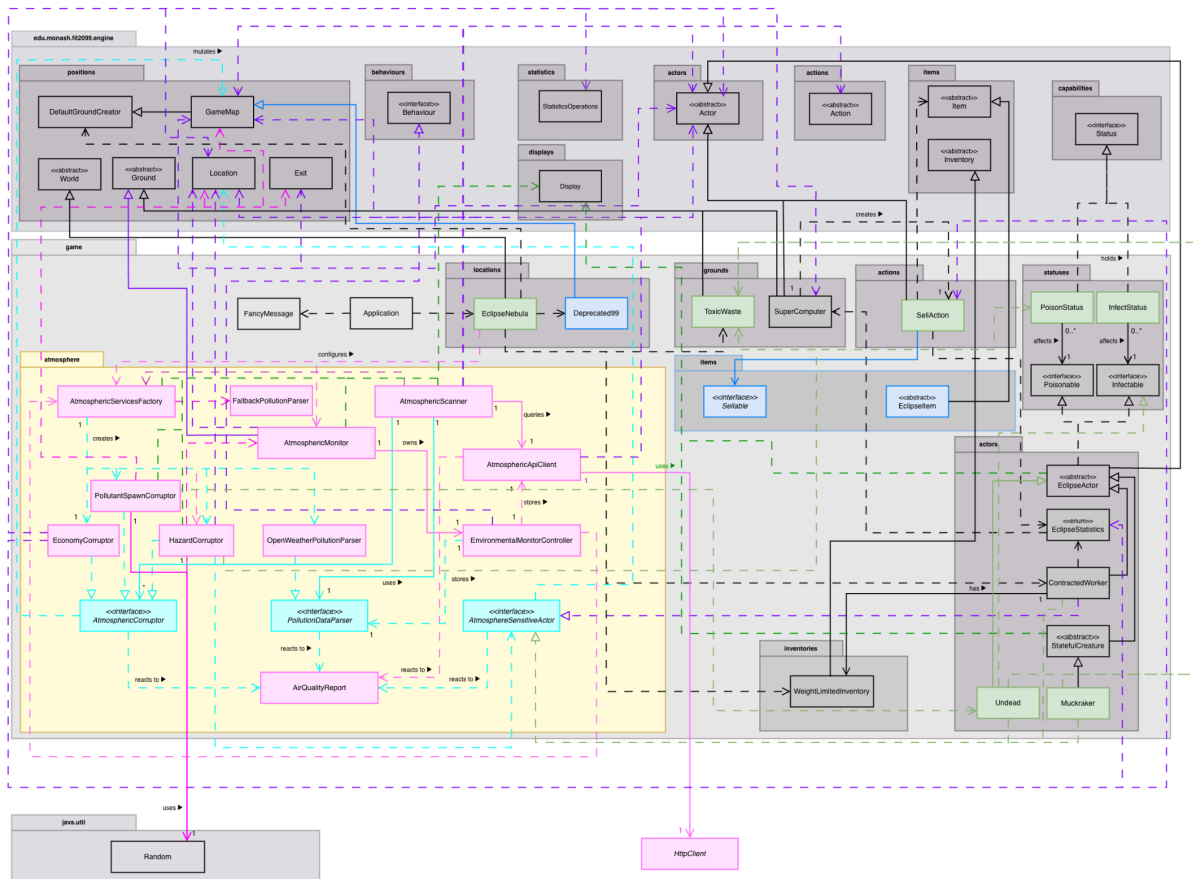


D. REQ 5 (Erwyna Soo Wen Xin, 36555789)

UML

The colour grouping reflects the five major design components of REQ5:

1. Dark Purple = atmosphere package to outsider packages
3. Light Blue = Unedited Used Classes
4. Green = Edited Used Classes
5. Pink = New Classes
6. Mint = Abstractions (a part of the new classes)



REQ 1

A.

Design 1: Quota Cycle [Alternative Design]

The initial design created a single shared QuotaManager instance in EclipseNebula, which was passed into every SuperComputer constructor and also stored as a field on EclipseNebula.

To find adjacent workers on the deadline, QuotaManager maintained a List<Location> superComputerLocations that was manually populated via registerSuperComputerLocation() calls in EclipseNebula after the maps were created.

EclipseNebula also overrode gameLoop() to call quotaManager.tick() each turn, and ContractedWorker stored a QuotaManager reference purely to display the quota HUD in playTurn().

(Note that this design doesn't meet the functional requirement either, as it was specified by staff that supercomputers are to have a separate quota.)

Pros	Cons
Single shared QuotaManager simple mental model, one source of truth for company credits	Connascence of values. The SuperComputer tile coordinates were hardcoded twice. Once in the map layout and again in registerSuperComputerLocation() calls in EclipseNebula. If a SuperComputer tile moved, both places had to be updated in sync, creating a fragile, tight coupling

HUD display integrated naturally in ContractedWorker.playTurn()	EclipseNebula held too many responsibilities. Creating QuotaManager, registering locations, overriding gameLoop(), and wiring workers with QuotaManager, it's a potential code smell
	ContractedWorker had an unnecessary dependency on QuotaManager. It only used it to print one line, which is not the worker's concern
	Chicken and egg problem because QuotaManager needed SuperComputer locations, but those locations only existed after the maps were created, forcing a specific initialisation order

Design 2: Quota Cycle [Chosen Design]

Each SuperComputer creates its own QuotaManager internally at construction time. The SuperComputer overrides tick(Location location). The engine already calls this every turn for all Ground objects and passes its own tile location to quotaManager.updateTurn(location). QuotaManager uses that location directly in fireAdjacentWorkers(), eliminating all need for location registration.

The quota HUD is printed inside QuotaManager.updateTurn() directly, so ContractedWorker no longer needs a QuotaManager reference at all. EclipseNebula no longer overrides gameLoop(), stores no QuotaManager field, and makes no registration calls.

Pros	Cons
Weaken connascence of values the SuperComputer tile location is provided by the engine naturally through tick(Location location). No coordinates are ever hardcoded twice	Each SuperComputer has its own independent quota in a multiplayer scenario with multiple SuperComputers across maps, workers on different maps contribute to different quotas rather than a single shared one. This could be a design concern if the spec ever requires a global quota (which in Ed forum is covered that each is individual instead of global)
Self-contained. Each SuperComputer contains its own quota cycle. Adding a new SuperComputer to any map automatically gets its own independent quota with no extra wiring	

Separation of concerns QuotaManager handles all quota logic, SuperComputer handles trading and ticking, ContractedWorker is no longer aware of the quota system at all (removed tight coupling)	
Avoids primitive obsession by separating quota state (credits, quota target, turns, rank) into QuotaManager rather than storing them as raw integers on SuperComputer, the design is cleaner and extensible.	

B.

Design 1: Plasma Cutter and Cuttable Interface [Alternative Design]

The initial CutAction stored a PlasmaCutter field alongside the Cuttable, and AluminiumDoor and Vent scanned adjacent tiles to find their own location using a loop over exits during cutBy().

PROS & CONS of Design 1

Pros	Cons
menuDescription() could reference the specific tool used	Forces tight coupling. CutAction knowing about PlasmaCutter specifically means adding a second cutting tool would require changes to CutAction (violates OCP)
	Location scanning in cutBy() was redundant, the location was already known at the point allowableActions() was called

Design 2: Plasma Cutter and Cuttable Interface [Chosen Design]

CutAction holds only a Cuttable and a Location, both passed at construction from allowableActions() which already has both. PlasmaCutter is not referenced by CutAction at all, its presence is detected via ItemAbilities.CUTTER, following the same pattern as ItemAbilities.UNLOCKER and ItemAbilities.STERILISER already established in the codebase. This avoids instanceof entirely (looser coupling).

Pros	Cons
Open/closed principle, adding a new cutting tool only requires enabling ItemAbilities.CUTTER on it, no changes to CutAction or any Cuttable	
No redundant location scanning in cutBy()	
Consistent with existing ability checking patterns in the codebase	

C.

Design 1: Industrial Fan (Sell Side Effect and SuperComputer Location)

[Alternative Design]

IndustrialFan stored superComputerLocation and a List<Spawner> as fields, requiring a parameterised constructor. Vent.cutBy() passed these in when creating the fan. A second no-arg constructor was needed for ScrapSnatcherSpawner (REQ-2), which creates fans from the depositable list without knowing the SuperComputer location, requiring a null guard in soldBy().

PROS & CONS of Design 1

Pros	Cons
IndustrialFan could spawn from any spawner in the list extensible in theory	Two constructors created ambiguity, example, IndustrialFan::new was ambiguous as a Supplier<Item> and required an explicit lambda () -> new IndustrialFan()
	Vent needed to know and store the SuperComputer location just to pass it to IndustrialFan, coupling two classes that shouldn't need to know about each other
	The null guard in soldBy() was defensive code masking a design inconsistency, the same class behaved differently depending on how it was constructed

Design 2: Industrial Fan (Sell Side Effect and SuperComputer Location)

[Chosen Design]

Sellable.soldBy() was updated to accept a Location superComputerLocation parameter, passed through from SellAction which receives it from SuperComputer.allowableActions(). IndustrialFan now has a single no-arg constructor and its soldBy() receives the SuperComputer location as a parameter rather than storing it as a field.

Pros	Cons
Single constructor, for REQ 2 IndustrialFan::new works unambiguously as a Supplier<Item>	All Sellable implementors must update their soldBy() signature even if they don't use superComputerLocation (a minor inconvenience offset by the overall cleanliness, so it was worth refactoring)
Vent no longer needs to know anything about SuperComputer location when dropping IndustrialFan	
Business rule is enforced structurally, the location is always provided at sell time	
Follows the same pattern as SellAction passing context to soldBy(), consistent with how CutAction passes Location to cutBy()	

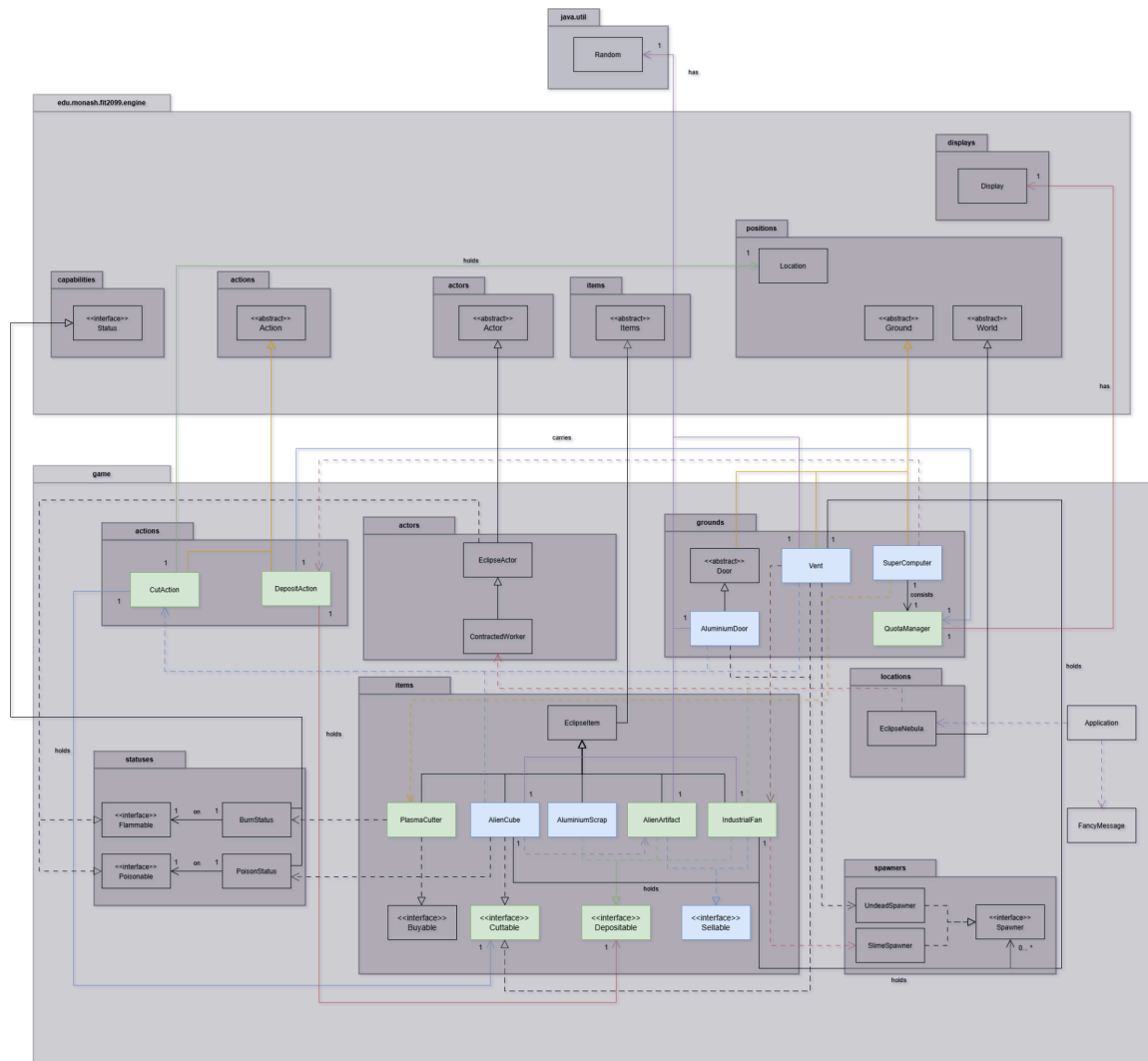
D.

Depositable Interface and DepositAction [Chosen Design]

Both designs were similar here. DepositAction mirrors SellAction, it handles the credit addition (common for all deposits) and delegates side effects to Depositable (as it varies). The key decision was keeping company credits inside QuotaManager rather than as an EclipseStatistics enum value on the actor.

Worker credits are per worker values stored as engine statistics, each worker has their own balance. Company credits are a global game state value belonging to no single actor, rather in a QuotaManager associated with a SuperComputer. Storing them in QuotaManager as a field rather than an actor statistic correctly models this distinction and avoids attaching company state to an entity that doesn't own it (separation of concerns).

Conclusion of REQ 1



Note:

Legend

Green represents: classes that are new
 Blue represents: classes that existed but has been modified
 White/Default represents: classes that has no changes to it

From the UML diagram above comparing both designs, ultimately, Design 2 is the final implementation chosen for REQ1. The key improvements over Design 1 are the elimination of connascence of values through `SuperComputer.tick()` providing its own location naturally, the removal of unnecessary coupling between `ContractedWorker` and `QuotaManager`, and the simplification of `IndustrialFan` to a single constructor with the `SuperComputer`'s location passed through `SellAction`.

These improvements reflect a deliberate application of SOLID principles throughout the design. SRP is upheld by extracting quota state and lifecycle logic into

QuotaManager, keeping SuperComputer focused solely on terminal interactions rather than tracking credits, ranks, and deadlines itself. OCP (open close principle) is honoured by the Depositable and Cuttable interfaces new items with novel deposit or cut behaviours can be introduced without modifying DepositAction, CutAction, or SuperComputer. DIP (dependency inversion) is evident in SuperComputer's catalogue, which depends on the Buyable abstraction rather than concrete item types, and in DepositAction depending on the Depositable interface rather than any specific item class.

DRY is respected throughout, deposit credit logic lives exclusively in DepositAction.execute() rather than being duplicated across every Depositable item, and cut delegation is handled once in CutAction rather than scattered across callers. Each SuperComputer owning its own QuotaManager makes the system self-contained and extensible, new SuperComputer instances on new maps automatically participate in the quota system with no additional wiring. By separating quota state into QuotaManager avoids primitive obsession and provides a clean extension point for future quota related mechanics without touching SuperComputer or any other class. This separation also promotes code reusability so other entities besides SuperComputer may use a QuotaManager in the future.

REQ 2

A.

Design 1: Behaviour-Composed Scrap Snatcher

ScrapSnatcher delegates wandering, hoarding, and combat to priority-ordered engine behaviours. SnatchBehaviour scans the actor's tile and returns PickupAction for the first item exposing the Depositable capability, so hoarding applies to any depositable resource without naming concrete item classes. WanderBehaviour supplies random movement. When a Parasite infects the snatcher via Infectable.infection(), SnatchBehaviour is removed and AttackBehaviour with BareFist (1 damage, 10% hit) is added, attack behaviour is worker-only targeting through WORKER_HOSTILE.

Hypothetically, a snatcher on a tile with Industrial Fan picks it up each turn until infected, then swings at adjacent workers. SRP keeps each behaviour cohesive; OCP lets new depositables join without editing SnatchBehaviour. DRY centralises pick-up logic. Connascence of position links integer priorities to execution order (moderate coupling); connascence of meaning ties Depositable to items (weak, deliberate). Smell: SnatchBehaviour omits an explicit null return when no item exists, risking unclear control flow. Extensibility is strong: adding CopperScrap only requires implementing Depositable. Extension: InfectedSnatcherProfile could associate with ScrapSnatcher to bundle weapon and behaviour swaps on infection.

Pros	Cons
Engine-aligned behaviour composition keeps actor logic thin and testable in isolation.	Priority integers create connascence of position when inserting new behaviours.
Capability-based snatching scales to future depositables without modifying hoarding code.	Infection transition logic is embedded in the actor rather than a reusable state object.

Infection side-effects stay local to ScrapSnatcher.infection(), preserving encapsulation	Missing explicit null return in SnatchBehaviour is a minor maintainability defect.
--	--

Design 2: Alternative Design

ScrapSnatcher could extend StatefulCreature like Muckraker, using HoardingState for wandering plus snatching and DefensiveState after infection. Snatching would live in state transition or immediateEffect rather than a dedicated behaviour. Hoarding and attack behaviours can be implemented by emotion swaps triggered when InfectStatus ticks.

Example: infection moves the snatcher from GREEDY to ANGRY, attaching attack behaviours automatically. DRY reuses the existing state hierarchy; Liskov substitution lets states plug into StatefulCreature. Connascence of execution is higher than the implemented design because state transitions must coordinate hoarding disablement and combat enablement in one place. This alternative reduces priority-integer smell but introduces more classes and indirection. Extension: InfectedState associates with StatefulCreature and collaborates with InfectStatus each tick.

Pros	Cons
Reuses proven State/Emotion infrastructure already in the game package.	Higher abstraction cost for a creature with only two behavioural modes.
Infection-driven transitions make attack behaviour visually explicit in the state graph.	Snatch logic split across state classes reduces cohesion compared to one behaviour.
Eliminates behaviour-priority connascence by centralising mode changes in states.	Tighter coupling to StatefulCreature limits reuse for simpler actors.

B.

Design 1: Injectable Spawner with Loot Explosion

ScrapSnatcherSpawner implements Spawner.spawnAt(), places a ScrapSnatcher, then lootExplosion drops one random depositable on each adjacent enterable tile via List<Supplier<Item>>. EclipseNebula registers AluminiumScrap, IndustrialFan, and AlienArtifact suppliers once; Deprecated99 and Overflow20 inject that list into Hole and Vent spawner pools, which spawn sources and loot explosion.

Example: a snatcher spawning from a Hole on 99-deprecated scatters up to eight random depositables nearby. DIP lets the spawner depend on Supplier<Item> not concrete classes; KISS keeps explosion logic inside one class. Connascence of type links the supplier list length to loot variety (moderate); connascence of name appears where both maps pass depositable into constructors (weak). Smell: each map repeats spawner-list assembly. Scalability: new depositables append one supplier in EclipseNebula only. Extension: LootTable could associate with ScrapSnatcherSpawner via composition to own random selection.

PROS & CONS of Design 1

Pros	Cons
Constructor injection makes the loot pool explicit and unit-testable with stub suppliers.	Map classes duplicate spawner-list wiring for holes, vents, and trees.
Reusing Spawner integrates snatchers into existing Hole and Vent spawn pipelines.	Supplier list connascence means an empty list would break explosion silently.
Future depositables extend the supplier list without editing explosion geometry.	Loot logic is tied to ScrapSnatcherSpawner rather than a shared scatter service.

Design 2: Alternative Design

A DepositableRegistry owned by EclipseNebula could expose randomDepositale() so every spawner queries one registry instead of receiving a list. Loot explosion would call the registry per adjacent tile. DRY removes repeated list passing; registry exhibits lower connascence of name across maps. Trade-off: hidden global dependency complicates isolated testing and obscures which maps share which pool versus explicit injection. Extension: DepositaleRegistry associates with World and is looked up by all Spawner implementations.

Pros	Cons
Single registration point minimises duplication across map constructors.	Global lookup increases coupling to EclipseNebula lifecycle.
Weaker connascence of name because maps no longer thread supplier lists.	Harder to give different maps distinct loot tables without registry branching.
Central registry simplifies adding depositables project-wide.	Extra abstraction layer for a small fixed set of three items.

C.

Design 1: Map-Injected Tree Growth Chain

FleshyTreeSprout and FleshyTreeMature implement Growable; GrowBehaviour drives timed probabilistic growth. Deprecated99.setTrees() wires Sprout directly to Mature then FleshyTreeMonolith, skipping sapling. Mature carries ScrapSnatcherSpawner; Monolith uses TreeTeleportBehaviour via Teleporter to warp adjacent workers, covering deprecated-map tree rules end to end.

Example: after 35 turns at 50% chance, Mature becomes Monolith and teleports nearby workers. OCP lets new stages join by constructor wiring, not editing grow(). Reusability: identical tree classes serve Overflow20 with different nextStage chains. Connascence of position is strong in nested constructor calls inside Deprecated99 (map knows exact topology); connascence of execution links GrowBehaviour statistics to each tree (moderate). Smell: deeply nested new expressions harm readability. Extension: TreeChainFactory in Deprecated99 could associate stage objects and return a configured Sprout.

PROS & CONS of Design 1

Pros	Cons
nextStage injection allows per-map growth paths without changing tree classes.	Nested instantiation in Deprecated99 creates strong positional connascence.
Growable plus GrowBehaviour reuse growth timing logic across all tree stages.	Map setup must know full stage graph, increasing configuration burden.

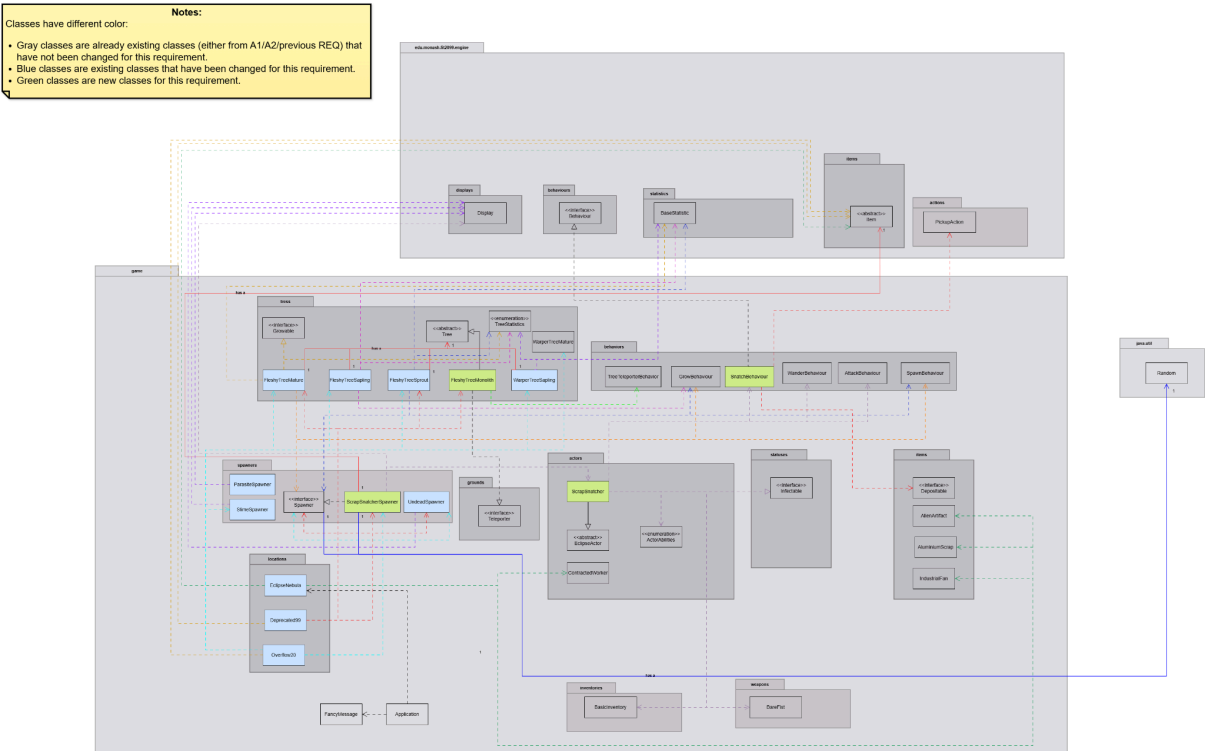
Monolith teleporter reuses Teleporter and TreeTeleportBehaviour from warper trees.	Long constructor chains are a readability smell when chains grow further.
--	---

Design 2: Alternative Design

Each tree could hardcode its successor inside `grow()`, as earlier `FleshyTreeSprout` did when it directly constructed `FleshyTreeSapling`. Deprecated-map rules would require conditional logic or subclasses per map inside tree classes. KISS yields fewer wiring classes; connascence of meaning inside trees rises because growth knowledge moves from maps into ground types, blocking the Sprout-to-Mature skip without editing tree source. Extension: override `grow()` in a `DeprecatedSprout` subclass depending on `Deprecated99`.

Pros	Cons
Growth transitions are visible inside each tree class without external wiring.	Map-specific rules force tree-class changes or subclass proliferation.
Fewer factory parameters simplify tree constructors on non-deprecated maps.	Hardcoded successors reduce reusability across 99-deprecated and 20-overflow.
Lower connascence of position in map setup classes.	Violates OCP when alters which stages exist per map.

Conclusion of REQ 2



The diagram above shows the final design of REQ 2, which utilizes Design 1 Behaviour Composition, Design 2 Injectable Spawner, and Design 3 Map-Injected Growth. All three implemented designs were retained because they align with engine conventions (KISS), share abstractions like Spawner and Growable (DRY), and honour SRP and OCP by localising hoarding, explosion, and growth concerns. Behaviour priorities and constructor injection keep connascence manageable and avoid state-machine or registry smells that would add classes without proportional benefit. Trade-offs include verbose tree wiring in Deprecated99 and repeated spawner assembly in maps. Stateful, registry, and hardcoded-growth alternatives would suit larger creature catalogues or many loot tables where central coordination outweighs the added abstraction.

REQ 3

A.

Design 1: Rideable Mount Lifecycle

Rideable in game.vehicles owns mount and dismount: MountAction removes the vehicle from the tile, places it in inventory, and applies RideStatus; DismountAction reverses each step. HoverBike, MechSuit, and AlienBeast inherit this lifecycle while supplying distinct tick effects and capability interfaces (Hoverer, Overclockable). REQ3 requires two abstractions and six complex cross-component effects; Rideable is the first abstraction, and its three subclasses spawn fire, collapse holes, or auto-consume ground each turn while mounted.

Example: mounting MechSuit grants VehicleAbilities.CRUSH and ignites adjacent tiles. SRP keeps mounting separate from vehicle combat; OCP adds vehicles via subclassing. DRY centralises inventory and status handling. Connascence of name ties RideStatus to mounted state (moderate); connascence of execution orders mount before ability checks (weak). Smell: non-portable items in inventory is semantically awkward, limiting reuse. Extensibility: new Rideable subclasses need no mount edits. Maintainability: lifecycle changes touch one class. Scalability: each vehicle adds one subclass. Extension: TandemRideable associates with two riders via shared RideStatus.

Pros	Cons
Single mount lifecycle reused by all three rideable concretes, reducing duplicated inventory logic.	Inventory storage for non-portable vehicles is an engine workaround that obscures intent.
RideStatus gives grounds and upgrades one reliable mounted flag without polling inventory contents.	Rideable cannot express vehicle-specific mount preconditions without subclass overrides.

Engine Action pattern keeps mount/dismount player-driven and testable.	Mount state depends on status presence, creating connascence of name with RideStatus.
--	---

Design 2: Alternative Design

A VehicleManager service could own mount state outside items: the actor references a mounted Rideable field, and MountAction delegates to the manager. Effects would still trigger from vehicle tick calls registered on the manager. Example: manager mounts HoverBike without placing it in inventory. KISS avoids inventory misuse; DIP lets grounds query the manager interface. Connascence of execution rises because manager, actor, and map must synchronise each turn, weaker name connascence than status checks. Trade-off: extra class versus clearer semantics. Extension: VehicleManager associates with Actor via composition and invokes Rideable.tick each turn.

Pros	Cons
Eliminates storing non-portable vehicles in inventory, improving semantic clarity.	Extra indirection layer beyond engine item conventions.
Central manager simplifies auditing which actor rides which vehicle.	Manager must mirror engine tick hooks, increasing integration surface.
Weaker connascence of name because mounted state is explicit, not inferred from status class.	Higher abstraction cost for three vehicles.

B.

Design 1: Conditional RideableUpgrade Abilities

RideableUpgrade is the second abstraction. Portable upgrades (BulldozerPlough, MagneticField, WarpBattery) register VehicleAbilities internally; each tick calls updateAbilities, enabling them on the carrier only when RideStatus is present. MagneticField additionally implements Igniter, corrupting prior tiles to ToxicWaste and offering IgniteAction; WarpBattery implements Teleporter for wall-piercing teleport with escalating PoisonStatus. Complex effects arise from upgrades modifying terrain, inventory, and actor statuses only while mounted.

Example: carrying MagneticField on foot does nothing; mounted, it vacuums adjacent items and poisons via ignite. ISP keeps ignite logic on Igniter only; Liskov substitution lets upgrades share ability toggling. DRY avoids repeating mount checks in each upgrade. Connascence of type links List<Enum<?>> to VehicleAbilities (moderate); connascence of meaning couples EXTRA_ENERGY to HoverBike hover blast (weak, intentional). Smell: ability propagation duplicates what Rideable already grants natively. Reusability: any future upgrade inherits toggle logic. Extension: ShieldUpgrade extends RideableUpgrade and registers a new ability without editing siblings.

PROS & CONS of Design 1

Pros	Cons
Conditional toggling prevents upgrades from affecting on-foot actors, matching design intent.	Per-turn ability enable/disable adds execution connascence with engine tick order.
Shared updateAbilities keeps three upgrade classes focused on their unique effects.	Enum<?> ability list sacrifices compile-time safety.

Composable upgrades stack with any Rideable, supporting varied loadouts.	Passive and active effects split across upgrade and action classes, raising navigation cost.
--	--

Design 2: Alternative Design

Upgrades could be Behaviour objects injected into the mounted Rideable at mount time rather than portable items. Rideable.mount would attach MagneticFieldBehaviour to the rider; dismount removes it. Effects execute inside behaviours returning actions. Example: WarpBatteryBehaviour offers teleport when mounted. OCP adds upgrades without new item subclasses; connascence of position links behaviour priority to execution (stronger than status gating). Trade-off: behaviours eliminate portable pickup but lose the carry-then-mount player loop. Extension: UpgradeBehaviour associates with Rideable through mount-time registration.

Pros	Cons
Removes per-turn ability toggling by attaching effects only while mounted.	Upgrades become non-portable behaviours, changing the carry-in-inventory mechanic.
Behaviour composition aligns with engine actor patterns used elsewhere in the game package.	Behaviour priority connascence is harder to reason about than a boolean mounted flag.
Weaker connascence of type because abilities need not propagate through Enum lists.	Retrofitting three item upgrades into behaviours requires wider refactoring.

C.

Design 1: Capability Interfaces with Action Delegation

Terrain and actors expose capabilities (Crushable, Bulldozeable, Purifiable) implemented by retrofitted Wall, Door, ToxicWaste, and EclipseActor. Ground allowableActions checks VehicleAbilities on the actor, then offers CrushAction, BulldozeAction, or PurifyAction, which delegate back to the capability method. Sand created by bulldozing adds terrain modification which state manipulation beyond numeric tweaks.

Example: a MechSuit rider crushes a Door into Dirt; a BulldozerPlough rider bulldozes Wall into slipping Sand. DIP lets actions depend on interfaces, not concrete grounds; OCP adds crushable targets without editing vehicles. KISS keeps each action as a thin wrapper. Connascence of meaning ties VehicleAbilities.CRUSH to Crushable targets (moderate); connascence of position requires ability checks in every ground allowableActions (moderate). Smell: repeated hasAbility guards across grounds duplicate gate logic. Extensibility: new Crushable grounds need only implement the interface. Extension: CrushableGate implements Crushable and associates with map tiles via Ground.

PROS & CONS of Design 1

Pros	Cons
Actions isolate player intent from terrain transformation logic.	Ability checks scattered across multiple ground classes violate DRY at the gate level.
New interaction types reuse the same capability-plus-action template.	Each capability needs a matching action class, increasing class count.

	Bulldoze geometry differs between Wall and EclipseActor, splitting similar logic.
--	---

Design 2: Alternative Design

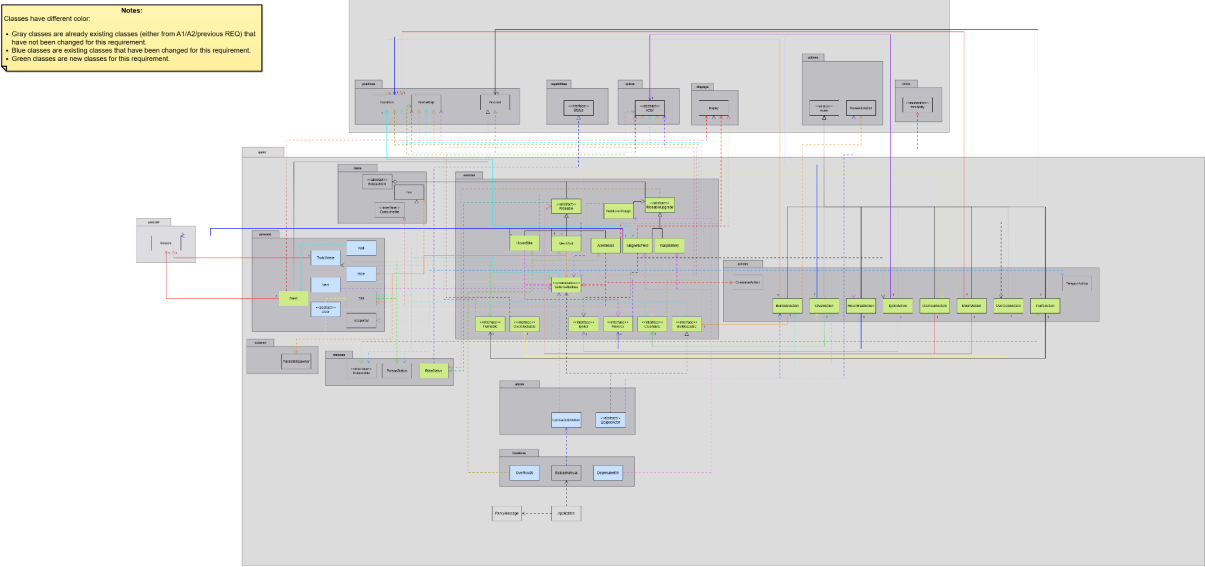
A `VehicleInteractionResolver` could centralise ability-to-action mapping: grounds call `resolver.actionsFor(actor, this)` instead of checking abilities locally. Crushing and bulldozing would register in one table. Example: resolver maps CRUSH plus `Crushable` to `CrushAction`. DRY removes repeated guards; connascence of position concentrates in the resolver (stronger single point). Trade-off: grounds lose local control over which actions appear. Extension: `VehicleInteractionResolver` associates with `GameMap` and queries actor abilities each turn.

Pros	Cons
Single registry eliminates duplicated <code>hasAbility</code> checks across grounds.	Resolver becomes a god-object as interaction types grow.
New vehicle-ground pairs register once without editing existing ground classes.	Grounds cannot easily customise action availability per tile type.
Lower connascence of position across individual ground files.	Indirection reduces readability compared to local <code>allowableActions</code> .

Comparison of Alternative Designs

The vehicle-manager alternative weakens name connascence versus RideStatus but adds a coordination class. The behaviour-based upgrade alternative trades per-tick ability toggling for priority connascence inside the behaviour stack. The interaction resolver best reduces coupling across grounds yet introduces a central registry whose complexity grows with each new ability pair. Among alternatives, the resolver minimises scattered coupling most aggressively, at the cost of abstraction and loss of ground-local action control.

Conclusion of REQ 3



The diagram above shows the final design of REQ 3, which utilizes Design 1 Rideable Mount Lifecycle, Design 2 Conditional RideableUpgrade Abilities, and Design 3 Capability Interfaces with Action Delegation. All three were kept because they compose cleanly: mount establishes RideStatus, upgrades gate abilities on that status, and capability interfaces let retrofitted grounds react without vehicle edits. KISS favours engine-native items and actions over a manager or resolver; DRY shares mount and ability-toggle logic centrally. SRP and OCP distribute vehicle, upgrade, and terrain concerns into separate extension points. Moderate name and meaning connascence around RideStatus and VehicleAbilities remains readable and avoids god-classes. Trade-offs include inventory-based mounting and repeated ability guards. Manager, behaviour, and resolver alternatives suit larger vehicle catalogues where central coordination outweighs added classes.

REQ 4

Decision 1a: Creating the turrets as per the proposal.

Chosen design: Create an abstract class Turret, which inherits Ground, and have the proposed turrets inherit Turret.

Advantages	Disadvantages
Allows standardizing the common behaviour of turrets (detecting enemies, determining when a projectile should be fired etc.)	Adds a new abstraction layer, increasing the number of layers the turrets are tightly coupled to (Turret, Ground etc.)

Alternative design: No Turret abstract class. Instead, the proposed turrets will just extend Ground directly.

Advantages	Disadvantages
Avoids adding a new abstraction layer, minimizing the number of layers each turret is tightly coupled with.	Common behaviour will need to be repeated across each turret concrete class.

The chosen design was used as each proposed turret shares common behaviour (e.g. detecting enemies) as well as common statistics (e.g. ammunition capacity). By having those standardized in the Turret abstract class, unnecessary code duplication, which is a code smell, is avoided, adhering to DRY. This makes it more maintainable as any changes to common behaviour will only need to be changed once rather than for each existing type of turret, which justifies having the additional abstraction layer.

Similarly, Projectile was also created as an abstract class with the same reasoning of mainly adhering to DRY.

Also, the Creative Mode requirement explicitly requires 2 abstractions, so the alternative design would've not even met the requirement.

Decision 1b: Should Projectile directly extend Item or EclipseItem?

The chosen design was to directly extend Item. EclipseItem's purpose is to standardize the game's items to have a weight and portable ability. Since Projectile is non-portable (as it's fired and moving) and does not need a weight, there's no point in extending EclipseItem just to remove the portable ability and make it 0 weight. By directly extending Item, this removes an unnecessary layer of abstraction as Projectile would not be tightly coupled with EclipseItem, which does adhere to KISS. This also avoids the code smell of Refused Bequest (as it would remove EclipseItem's portability), as well as another code smell of having too deep inheritance.

Decision 2a: Placing numerical values of statistics and effects in the classes of this requirement (e.g. GunTurret's ammo capacity and FireBullet's velocity).

The chosen design was to define those values as static final constants (e.g. AMMO_CAPACITY, VELOCITY and HIT_DAMAGE). The alternative design would be to simply not have those constants and have numeric literals instead (e.g. actor.hurt(2) instead of actor.hurt(HIT_DAMAGE)).

The chosen design was used as it eliminates magic numbers, weakening the connascence of meaning into connascence of name. Furthermore, this avoids the code smell of having excessive literals/magic numbers. This would improve readability especially for other, new people working on the code since these values now have a meaning attached to them. This also improves maintainability in case the same value (e.g. HIT_DAMAGE) would need to be used multiple times in the class. If said value would need to be changed, it only needs to be changed once (just change the value of the constant) rather than needing to find and change every occurrence of it.

Decision 2b: Structuring the tick method of Turret.

Chosen design: Split different parts of the logic (detecting enemies, determining if ready to fire etc.) into their own method.

Advantages	Disadvantages
This would be more readable.	May need to be more careful about the order in which methods are called (e.g. <code>getTargetDestination</code> must be called before <code>fireProjectileAt</code>).
Allows subclasses to override some methods (e.g. extra conditions for <code>isReady</code> method) without having to rewrite logic for the other methods.	-

Alternative design: Keep all of the logic in the tick method.

Advantages	Disadvantages
Seems simpler to just keep all of the logic in one method.	Less readable and maintainable as making changes would require finding the specific part of the logic in the long method to change.
-	Requires rewriting the entire method if a subclass needs to override it.

The chosen design was used as it breaks apart a long method (which in of itself is a code smell) into multiple methods with their own clear responsibility (e.g. detecting an enemy). This adheres to the SRP. As it is more modular, it improves readability and maintainability as it is easier to find a part of the logic to be changed. Subclasses that need to override some (e.g. isReady) but not all of the methods may also do so without needing to rewrite logic for the unchanged methods, promoting extensibility and adhering to DRY as well.

While there is a connascence of execution (call `getTargetDestination` before `fireProjectileAt`), the Turret abstract class ensures that it will execute in this order, and its subclasses can simply call super tick method to use this. They do not need to handle calling these methods and ensuring the correct order again, which avoids potential errors.

Decision 3: Who should be moving Projectiles?

Chosen design: Projectiles will move themselves when their tick method is called every turn.

Advantages	Disadvantages
Each Projectile is clearly responsible for their own movement.	Based on how the GameMap ticks each location, after moving some tiles, a projectile the GameMap may still call tick on it again, in the same turn, causing it to move again. This unintended behaviour must be addressed.

Alternative design 1: The Turret that fired the Projectile will handle its movement every turn too.

Advantages	Disadvantages
Easy way to ensure the GameMap doesn't cause projectiles to move again in the same turn, since the GameMap will only call Turret's tick method once.	If the Turret ground gets replaced, GameMap will no longer call its tick method. In turn, its projectiles fired will no longer be moved and is just stuck there permanently. This is unintended behaviour.
-	Turret also has an additional responsibility of handling the movement of its projectiles.

Alternative design 2: Projectiles don't have to move. They just hit their target instantly, so they don't even appear on the map. (This will not be discussed further, as explained in its disadvantage)

Advantages	Disadvantages
Would not have any issues from the above designs.	Removes the velocity component of projectiles, which as a feature would be less interesting. This is why it was proposed to have variable velocity rather than just instant hit.

The chosen design was used as Projectiles would be clearly responsible for moving themselves, acting independently from the entity that fired it. This adheres to the SRP. If the game was extended to have, for example, player-held guns, they will just need to spawn a projectile with a set destination, without needing to duplicate code for handling projectile movement, adhering to DRY as well. Furthermore, this avoids the Feature Envy code smell the alternative design has. In the alternative design, Turret would have a moveProjectile method that excessively interacts with Projectile's methods, like isStopped and getCloserDestination. This is a sign that the logic of moving projectiles should belong to Projectile itself.

Note: Since there is unintended behaviour for this design, it needs to be addressed. Because of how the GameMap calls tick on each location, if a Projectile was moved anywhere downwards, OR in the same y-coordinate, anywhere rightwards, the GameMap will call the Projectile's tick method (causing it to move) again. As a workaround, if the Projectile detects that it moved in such a way, it will "lock" itself, so that when tick is called, all it does is "unlock" itself without executing the rest of the logic. Since the engine itself must not be changed, this is to address the limitation of the game engine while keeping the proposed feature more interesting.

REQ 5 (Erwyna)

A. Atmospheric API Client and Parser Strategy

Design 1: AtmosphericApiClient with Parser Strategy and Fallback (Implemented Design)

AtmosphericApiClient is responsible only for building and executing the OpenWeather Air Pollution API request using the atmospheric anchor location from the scan pipeline and an environment variable API key. It delegates JSON parsing to a parser abstraction from AtmosphericServicesFactory, so the client does not need to know the response format. OpenWeatherPollutionParser handles the real API payload, while FallbackPollutionParser produces safe AirQualityReport instances when no key is available.

The API key is read at runtime via `System.getenv("OPENWEATHER_API_KEY")`, and when that variable is absent or blank, the factory selects `FallbackPollutionParser` so the feature still runs safely without exposing secrets or hard-coding credentials.

PROS & CONS of Design 1

Pros	Cons
Uses game state (monitor coordinates) to build a dynamic URL, satisfying the REQ 5 meaningful-impact rule.	Requires maintaining OpenWeather-specific parsing logic in a parser class, so client and parser must stay in sync with schema changes.
Separates concerns: AtmosphericApiClient only fetches, while parsers only convert JSON into AirQualityReport.	Adds a fallback versus real parser decision, which slightly increases setup complexity compared to a single direct parser.
Supports the Open-Closed Principle because new parser types can be added without modifying the client or corruptors.	Requires clear documentation so markers know how to set the environment variable and what happens when it is missing.
Keeps security local by reading the API key via System.getenv and never hard-coding it.	

Design 2: Single Hardcoded Client That Parses JSON Inline (Alternative Design)

In the alternative design, the client would both make the HTTP request and parse the JSON in one large method, then build the AirQualityReport and notify the corruptors directly.

Pros	Cons
Simpler to trace at first glance because HTTP calls and JSON parsing live in one place.	Couples the client tightly to the OpenWeather schema, so any JSON change forces edits in the same class, violating the Open-Closed Principle.
	Makes it harder to add a fallback or stub parser, limiting offline demos and tests.

The engine's Behaviour<T, R> abstraction is not used for the monitor because it does not choose between actions. AtmosphericMonitor is environmental infrastructure, so it extends Ground and uses tick(Location) to trigger scans each turn. Design 1 is used because it separates fetching from parsing, lets us swap parser strategies without changing the client or corruptors, and meets REQ 5 security rules by keeping the API key in the environment with a safe fallback.

B. AtmosphericCorruptors as API-Driven Effect Engines

Design 1: Separate AtmosphericCorruptor Implementations for Hazard and Economy Effects (Implemented Design)

The toxic atmosphere feature uses separate AtmosphericCorruptor implementations to translate AirQualityReport into game effects. HazardCorruptor applies AQI-tier hazard mechanics, EconomyCorruptor applies SO2-driven shop disruption, and PollutantSpawnCorruptor handles severe-AQI enemy spawning near the atmospheric anchor.

Together, these corruptors show that one API report can drive multiple independent world effects through the same AtmosphericCorruptor abstraction.

PROS & CONS of Design 1

Pros	Cons
Keeps hazard and economy concerns separated, improving cohesion and making each class easier to understand.	Introduces more small classes, so the feature has more files and readers must jump between corruptors to see all atmosphere effects.
Lets each corruptor evolve independently, e.g. new hazard shapes without touching economy logic.	Requires coordination between hazard and economy effects at certain AQI tiers and adds more moving parts to explain in the design rationale and README.
Reduces duplicate parsing or API logic because both corruptors share the AirQualityReport abstraction.	
Supports DIP because high-level map behaviour depends on AirQualityReport rather than the API client.	

Design 2: Single AtmosphericManager Handling All Map and Economy Effects (Alternative Design)

In this alternative, a single AtmosphericManager class would receive AirQualityReport and directly mutate terrain, actors, and economy in one large method, applying all AQI effects in one long chain of conditionals.

Pros	Cons
Provides one central place where all atmosphere-driven effects can be inspected.	Quickly becomes a god class with many responsibilities, violating the Single Responsibility Principle.
Easier to wire into the game initially, since only one type needs to be registered.	Mixes unrelated concerns like terrain mutation, actor health, and credit changes into one class, increasing coupling and maintenance cost.

Design 1 is used because splitting hazard and economy into separate corruptors keeps each responsibility focused and easier to extend while still delivering cross-component interactions. It avoids a god class, aligns with SRP and DIP, and makes the mapping from API data to map and economy changes clearer.

C. AtmosphereSensitiveActor and Actor-Level Effects

Design 1: AtmosphereSensitiveActor Capability with Actor-Controlled Effects (Implemented Design)

Atmospheric effects on actors are modelled through AtmosphereSensitiveActor, which lets any actor declare how it handles an AirQualityReport. HazardCorruptor iterates over the map, finds actors with this capability, and calls applyAtmosphere(report, Location here) so each actor owns its damage, poison, or movement reactions instead of relying on type checks in the corruptor. The monitor's map Location, supplied by the engine on each tick, is what every downstream class (API client, scanner, corruptors) uses as the centre point for atmosphere effects. This avoids introducing an empty marker interface just to identify the monitor, since the Location itself already carries the same information without any extra type.

PROS & CONS of Design 1

Pros	Cons
Removes the need for type checks or downcasting in the corruptor.	Requires each atmosphere-sensitive actor to implement <code>applyAtmosphere</code> , adding code to multiple classes.
Supports the Open-Closed Principle because new actors can join by implementing the interface instead of editing the corruptor.	Spreads atmosphere logic across several actor classes, so readers must navigate multiple files and the design needs clear documentation about which actors are atmosphere-sensitive.
Improves reuse and reduces connascence of type, since the corruptor talks to the shared capability rather than concrete actor classes.	

Design 2: Corruptor-Driven Effects with Direct Actor Type Checks (Alternative Design)

In the alternative design, HazardCorruptor would inspect each actor's runtime type and apply hard-coded effects directly, with no shared capability interface and actors unaware of the atmosphere system.

Pros	Cons
Puts all atmosphere-related actor effects in one class, which can look centralised and easy to scan at first.	Breaks the unit's requirement to avoid instanceof and downcasting and increases coupling between the corruptor and each actor.
Avoids adding new methods to actor classes, keeping those types smaller initially.	Violates the Open-Closed Principle, since every new actor type requires editing the same corruptor file.

Design 1 is used because it keeps the atmospheric system polymorphic and future-proof. Actors opt into the behaviour through AtmosphereSensitiveActor, and the corruptor depends only on the abstraction instead of concrete classes, which reduces connascence of type between them. This matches the unit's abstraction and dependency inversion guidelines and avoids code smells like long instanceof chains.

D. AQI Tiers, Probabilities, and Map-Level Complexity

Design 1: Three-Tier AQI System with Probabilistic Map Effects (Implemented Design)

The feature interprets the OpenWeather 1-5 AQI scale using three tiers: mild, moderate, and severe, with each tier driving different guaranteed and probabilistic effects. Moderate AQI creates local toxic puddles around sensitive actors with a 25% per-tile chance, while severe AQI adds a border ring and hotspot corruption within Manhattan distance 2 with a 50% spread chance. AQI thresholds and key values are stored as named constants to avoid magic numbers and make tuning easier.

PROS & CONS of Design 1

Pros	Cons
Keeps the design simple with three clear tiers instead of many small ranges.	Adds several probability checks per tick, which need careful balancing to avoid overly punishing or trivial gameplay.
Encodes meaningful complexity by mixing guaranteed actor effects with map-level randomness such as puddles, borders, and hotspots.	Requires careful documentation of percentages and thresholds and adds more code paths to test, especially when AQI changes between turns.
Uses named constants to remove magic numbers, improving maintainability.	Relies on tuning via code changes rather than an external configuration file, which may be less flexible.

Design 2: Direct Linear Mapping from AQI to Simple Damage (Alternative Design)

An alternative would be to ignore tiers and directly translate AQI into fixed damage or a single scalar applied to actors without terrain changes or probability. For example, actors might simply lose health each tick based on AQI, and no additional map corruption or economy disruption would occur.

Pros	Cons
Very simple to implement and understand, with a single formula linking AQI to damage.	Fails the meaningful game state impact and complexity rules because it only adjusts health values.
Easy to test, as there is only one main effect to verify.	Provides little cross-component interaction, which is expected for a stronger HD-level feature.
	Produces a less interesting and less visible feature in gameplay, making it weaker for the HD requirement.

Design 1 is used because it provides visible, multi-layered impact on actors and the map while still being simple enough to reason about in tiers. It combines actor capabilities, map mutation, and economy disruption without magic numbers or monolithic classes, keeping the design aligned with DRY, KISS, and SOLID.

Conclusion of REQ 5

The implemented design separates the toxic atmosphere feature into small, focused components: AtmosphericApiClient builds the dynamic API request, parsers produce AirQualityReport, corruptors apply map and economy effects, and atmosphere-sensitive actors handle their own reactions. The monitor is modelled as a Ground tile using tick(Location) rather than an Actor behaviour, which fits fixed environmental infrastructure. New parsers, corruptors, and actors can be added by extending existing abstractions without rewriting core classes, which improves maintainability and keeps the feature aligned with DRY, KISS, and SOLID while delivering meaningful API-driven changes to actors, terrain, the game economy, and severe-pollution enemy spawning.

